

ENTREGA DE TAREAS

Sprint 2: Navegación — FutbolWebApp

Pablo Arrazola Trebol

Prácticas — Data Control

13/11/2026

Resumen

Se han implementado en FutbolWebApp los cuatro tipos de navegación del estilo Mobile/Touch. Para cada caso se indica quién es la InvokingView, el InvokingObject, la InvokedView, cómo queda la pila de vistas (ViewStack) y qué eventos se disparan. Se ha añadido un evento OnNavigateForward en las vistas destino que muestra el tipo de navegación recibido, para comprobarlo en cada prueba.

1. Conceptos de navegación

Concepto	Significado
DrillDown	Navegar desde lo más general (Dashboard, listas) hasta el registro concreto (Zoom).
Mobile/Touch	Estilo de WebApp con vistas simples a pantalla completa entre las que se va avanzando.
Breadcrumb	Migas de pan: muestran la ruta recorrida (dónde estoy y de dónde vengo) y permiten volver.
ViewStack	Pila (LIFO) con las vistas abiertas. NavigateForward apila una vista; NavigateClose y NavigateCancel la desapilan.

Participantes

Participante	Qué es
InvokingView	Vista desde la que se navega (la que queda debajo en la pila).
InvokingObject	Objeto que lanza la navegación: una lista, un botón, un campo con prompt…
InvokedView	Vista destino que se apila y se muestra.
InvokedObject	Objeto de la vista destino que responde al volver (callback en la navegación hacia atrás).
ViewStack	La pila de vistas abiertas que gestiona el framework.

Tipos de navegación (eNavigateType)

El framework compara el DD del InvokingObject con el Main_DD de la vista destino y decide el tipo automáticamente:

Tipo	Relación entre DDs	Qué hace el framework
nfFromMain	Mismo DD (misma tabla)	La vista destino busca el mismo registro. Típico: Select → Zoom.
nfFromChild	El invocante es hijo del destino	La Select del padre se posiciona en el padre actual y, al elegir, actualiza el hijo. Típico: Zoom hijo → Select padre.
nfFromParent	El invocante es padre del destino	La vista destino queda restringida (relates-to) a los hijos del registro padre. Típico: Zoom liga → equipos.
nfUndefined	Sin relación entre DDs	No hace nada automático; el desarrollador pasa y recoge los datos. Típico: Dashboard, tablas sin

Tipo	Relación entre DDs	Qué hace el framework
		relación directa.

Modos de navegación

- NavigateForward: apila la vista destino con intercambio de información (tWebNavigateData, búsqueda automática según el tipo).
- NavigateForwardCustom: apila la vista sin intercambio automático de información.
- NavigateBegin (menú hamburguesa): vacía la pila, deja el Dashboard y abre la vista elegida; sin intercambio de información.
- Para volver: NavigateClose (vuelve y actualiza la vista anterior) y NavigateCancel (vuelve sin actualizar, es lo que hace el breadcrumb).

Eventos de navegación

Evento	Se envía a	Para qué
OnGetNavigateForwardData	InvokingObject	Preparar los datos que viajan: bReadOnly, bNewRecord, bSaveBeforeNavigate, NamedValues.
OnNavigateForward	InvokedView	Ajustar la vista según cómo se ha llegado (tipo, vista y objeto que la invocaron).
OnGetNavigateBackData	InvokedObject (vista que se cierra)	Preparar los datos que se devuelven al volver.
OnNavigateBack	InvokingObject	Recoger lo devuelto; imprescindible en nfUndefined, donde no hay actualización automática.

2. From main: Select → Zoom

En SelectEquipo, al pulsar una fila se abre ZoomEquipo con ese equipo.

```
// SelectEquipo.wo  (lista oList)
Procedure OnRowClick String sRowID
   Send NavigateForward of oZoomEquipo Self
End_Procedure
```

Participante	En este caso
InvokingView	oSelectEquipo
InvokingObject	oList, la cWebList de la Select (por eso se pasa Self desde OnRowClick)
InvokedView	oZoomEquipo
Tipo	nfFromMain: el DD de oList (oEquipo_DD) es de la misma tabla que el Main_DD del Zoom

¿Qué pasa con el InvokingObject? Es la lista. Su DDO tiene ya en buffer la fila pulsada; el framework guarda su RowID en NavigateData.sRowID y el Zoom busca

ese mismo registro. La lista no se destruye: queda en la vista de debajo de la pila y, al volver con NavigateClose, se refresca para mostrar los cambios guardados en el Zoom. Si el InvokingObject fuese un botón «Nuevo», bastaría con poner bNewRecord a True en su OnGetNavigateForwardData para abrir el Zoom vacío:

```
Procedure OnGetNavigateForwardData tWebNavigateData ByRef NavigateData Handle hoToView
   Move True to NavigateData.bNewRecord
End_Procedure
```

Comprobación del tipo recibido en las vistas destino:

```
// En cada vista destino (Zoom/Select)
Procedure OnNavigateForward tWebNavigateData NavigateData Handle hoInvokingView Handle
hoInvokingObject
   String sTipo
   Case Begin
      Case (NavigateData.eNavigateType = nfFromMain)
         Move "FromMain" to sTipo
         Case Break
      Case (NavigateData.eNavigateType = nfFromChild)
         Move "FromChild" to sTipo
         Case Break
      Case (NavigateData.eNavigateType = nfFromParent)
         Move "FromParent" to sTipo
         Case Break
      Case Else
         Move "Undefined" to sTipo
   Case End
   Send ShowInfoBox ("Navegación: " + sTipo)
End_Procedure
```

Pila de vistas:

```
Dashboard → SelectEquipo → ZoomEquipo
```

Captura: ZoomEquipo abierto desde la lista, con el aviso «Navegación: FromMain»:

3. From child: Zoom hijo → Select padre

En ZoomEquipo, el campo Liga tiene prompt. Al pulsarlo se abre SelectLiga posicionada en la liga actual del equipo; al elegir otra fila, se vuelve al Zoom con la liga cambiada.

```
// ZoomEquipo.wo
Object oLigaNombre is a cWebForm
   Entry_Item Liga.Nombre
   Set psLabel to "Liga"
   Set pbPromptButton to True

   Procedure OnPrompt
      Send NavigateForward of oSelectLiga Self
   End_Procedure
End_Object
```

```
// SelectLiga.wo  (lista oList)
Procedure OnRowClick String sRowID
   tWebNavigateData NavigateData
   Get GetNavigateData to NavigateData
   If (NavigateData.eNavigateType = nfFromChild) Begin
      Send NavigateClose Self        // devuelve la liga elegida
   End
   Else Begin
      Send NavigateForward of oZoomLiga Self
   End
End_Procedure
```

Participante	En este caso
InvokingView	oZoomEquipo
InvokingObject	oLigaNombre (campo Liga.Nombre del Zoom)
InvokedView	oSelectLiga
Tipo	nfFromChild: el campo está ligado a Liga, que es padre de Equipo en el DD del Zoom

Al hacer NavigateClose, el framework coloca la liga elegida en el DD padre del Zoom: el equipo queda apuntando a la nueva liga y basta con guardar. No hace falta OnNavigateBack, porque en nfFromChild la actualización es automática.

```
Dashboard → SelectEquipo → ZoomEquipo → SelectLiga  (NavigateClose: vuelve a ZoomEquipo)
```

Captura: SelectLiga abierta desde el prompt del ZoomEquipo («Navegación: FromChild»):

4. From parent: Zoom liga → equipos de esa liga

1.º Botón en ZoomLiga que muestra los equipos de esa liga. El botón navega a SelectEquipo. Como Liga es padre de Equipo, el tipo es nfFromParent y la lista sale ya filtrada por la liga del Zoom, sin escribir ningún filtro.

```
// ZoomLiga.wo
Object oVerEquipos is a cWebButton
   Set psCaption to "Ver equipos"
   Set piColumnSpan to 4

   Procedure OnClick
      Send NavigateForward of oSelectEquipo Self
   End_Procedure

   // Guarda la liga antes de salir (si es nueva, necesita existir)
   Procedure OnGetNavigateForwardData tWebNavigateData ByRef NavigateData Handle hoToView
      Move True to NavigateData.bSaveBeforeNavigate
   End_Procedure
End_Object
```

```
WebRegisterPath ntNavigateForward oSelectEquipo
```

Participante	En este caso
InvokingView	oZoomLiga
InvokingObject	oVerEquipos (el botón; su DD es el de la vista, oLiga_DD)
InvokedView	oSelectEquipo
Tipo	nfFromParent: Liga es padre de Equipo; SelectEquipo queda restringida a los equipos de esa liga

```
Dashboard → SelectLiga → ZoomLiga → SelectEquipo (solo equipos de la liga)
```

En SelectEquipo se puede aprovechar OnNavigateForward para cambiar el título a «Equipos de LaLiga» cuando NavigateData.eNavigateType = nfFromParent.

Captura: botón «Ver equipos» en ZoomLiga y SelectEquipo filtrada por LaLiga:

2.º Sin botón: los equipos dentro del propio ZoomLiga. Se añade al Zoom un DD de Equipo cuyo servidor es el DD de Liga, restringido a la liga actual, y una cWebList enlazada a él. Al moverse de liga, la lista cambia sola.

```
// ZoomLiga.wo  (estructura de DDs)
Object oLiga_DD is a cLigaDataDictionary
End_Object

Object oEquipo_DD is a cEquipoDataDictionary
    Set DDO_Server to oLiga_DD
    Set Constrain_File to Liga.File_Number    // solo equipos de la liga actual
End_Object

Set Main_DD to oLiga_DD
Set Server  to oLiga_DD

// Dentro del panel, debajo de los campos de la liga
Object oEquiposList is a cWebList
    Set Server to oEquipo_DD
    Set piHeight to 220
    Set pbServerOnRowClick to True
    Set psPlaceHolder to "Esta liga no tiene equipos"

    Object oEquipoNombre is a cWebColumn
        Entry_Item Equipo.Nombre
        Set psCaption to "Equipos de la liga"
    End_Object

    // Pulsar un equipo abre su ficha (nfFromMain: mismo DD que ZoomEquipo)
    Procedure OnRowClick String sRowID
        Send NavigateForward of oZoomEquipo Self
    End_Procedure
End_Object
```

```
WebRegisterPath ntNavigateForward oZoomEquipo
```

Aquí ya no hay navegación para ver los equipos: la relación padre-hijo se resuelve dentro de la misma vista con Constrain_File. Sí hay navegación al pulsar un equipo, y es nfFromMain porque la lista usa el DD de Equipo, igual que ZoomEquipo.

Captura: ZoomLiga con la lista de sus equipos integrada:

5. Undefined: Zoom jugador → Select posición

¿Hay relación entre jugador y posicion? No directa. Ninguna de las dos tiene clave ajena hacia la otra: la relación es de muchos a muchos y pasa por la tabla intermedia jugadorposicion (JugadorID, PosicionID), de la que ambas son padres. Como el DD del Zoom (jugador) y el de la Select (posicion) no son el mismo ni padre/hijo uno del otro, el framework clasifica la navegación como **nfUndefined** y no hace nada automático: ni filtra ni actualiza al volver. Esa lógica la pone el desarrollador.

Se usa la navegación para asignar una posición al jugador: el botón abre SelectPosicion, se elige una fila y, al volver, OnNavigateBack guarda el par jugador-posición en jugadorposicion.

```
// ZoomJugador.wo  (DD de la tabla intermedia, hija de jugador y posicion)
Object oPosicion_DD is a cPosicionDataDictionary
End_Object

Object oJugadorPosicion_DD is a cJugadorPosicionDataDictionary
   Set DDO_Server to oJugador_DD
   Set DDO_Server to oPosicion_DD
End_Object

Object oAsignarPosicion is a cWebButton
   Set psCaption to "Asignar posición"
   Set piColumnSpan to 4

   Procedure OnClick
      Send NavigateForward of oSelectPosicion Self
   End_Procedure

   Procedure OnGetNavigateForwardData tWebNavigateData ByRef NavigateData Handle hoToView
      Move True to NavigateData.bSaveBeforeNavigate
   End_Procedure

   // Al volver: NavigateData.sRowID es el registro elegido en SelectPosicion
   Procedure OnNavigateBack Handle hoCallback tWebNavigateData NavigateData
      RowID riPosicion
      Move (DeserializeRowID(NavigateData.sRowID)) to riPosicion
      Send Clear of oJugadorPosicion_DD
      Send FindByRowID of oPosicion_DD Posicion.File_Number riPosicion
      Send Request_Save of oJugadorPosicion_DD
   End_Procedure
End_Object
```

```
WebRegisterPath ntNavigateForward oSelectPosicion

// SelectPosicion.wo
// Desde el Dashboard también llega como nfUndefined: se distingue por la
// vista que la invoca y se guarda en una web property de servidor.
{ WebProperty=Server }
Property Boolean pbModoAsignar False

Procedure OnNavigateForward tWebNavigateData NavigateData Handle hoInvokingView Handle
hoInvokingObject
   WebSet pbModoAsignar to (hoInvokingView = oZoomJugador)
End_Procedure

// (lista oList)
Procedure OnRowClick String sRowID
   Boolean bAsignar
   WebGet pbModoAsignar of oSelectPosicion to bAsignar
   If bAsignar Begin
      Send NavigateClose Self     // devuelve la posición al jugador
   End
   Else Begin
      Send NavigateForward of oZoomPosicion Self
   End
End_Procedure
```

Participante	En este caso
InvokingView	oZoomJugador
InvokingObject	oAsignarPosicion (recibe OnNavigateBack al volver)
InvokedView	oSelectPosicion
Tipo	nfUndefined: sin relación directa entre los DDs de jugador y posicion

```
Dashboard → SelectJugador → ZoomJugador → SelectPosicion (NavigateClose:
OnNavigateBack guarda la posición)
```

La restricción única uqJugadorPosicion (JugadorID, PosicionID) impide asignar dos veces la misma posición al mismo jugador: el guardado devuelve un error de clave duplicada, que se puede mostrar al usuario.

Captura: SelectPosicion abierta desde ZoomJugador («Navegación: Undefined»):

Captura: registro creado en jugadorposicion (MySQL Workbench o Table Explorer):

6. Resumen de los casos

Caso	Desde (InvokingObject)	Hacia (InvokedView)	Tipo y efecto
From main	oList de SelectEquipo	ZoomEquipo	nfFromMain: busca el mismo equipo
From child	Campo Liga de ZoomEquipo	SelectLiga	nfFromChild: elige la liga y actualiza el equipo
From parent 1.º	Botón «Ver equipos» de ZoomLiga	SelectEquipo	nfFromParent: solo equipos de esa liga
From parent 2.º	— (lista dentro de ZoomLiga)	—	Sin navegación: Constrain_File en la misma vista
Undefined	Botón de ZoomJugador	SelectPosicion	nfUndefined: datos a mano con OnNavigateBack

Control de versiones: cada caso se ha desarrollado en su rama (feature/nav-from-main, feature/nav-from-child, feature/nav-from-parent, feature/nav-undefined), integrada en develop con merge --no-ff y publicada en main con la etiqueta v2.0-sprint2.